

Python Complete Handbook for Data Analytics — From Zero to Job-Ready

Python Complete Handbook for Data Analytics

From Zero to Job-Ready — Everything a DA Fresher Actually Needs

How to Use This Handbook

This is not a course. This is a reference built from real analyst work.

Read it once end to end. Then come back to each section when you need it.

Every concept here has shown up in real DA interviews or real client work.

Section 1 — Why Python for Data Analytics

Python is not a programming language for data analysts. It is a data tool that happens to use code.

You do not need to think like a software engineer. You need to think like an analyst who uses Python to work faster.

The entire skill set a DA needs in Python fits inside 3 libraries:

- **Pandas** — work with data tables
- **Matplotlib / Seaborn** — visualize data
- **NumPy** — numerical operations (used mostly behind the scenes)

Everything else is secondary. Master these 3 and you are interview ready.

Section 2 — Python Basics Every Analyst Must Know

2.1 Data Types

Type	Example	When You See It
int	25, 1000	Age, count, quantity
float	25.5, 10000.75	Price, salary, percentage
string	'Rahul', 'Delhi'	Names, cities, categories
boolean	True, False	Flags, conditions
NaN	NaN	Missing values in datasets

```
# Check data type
type(25)          # int
type(25.5)        # float
type('Rahul')    # str
type(True)        # bool
```

2.2 Lists and Dictionaries

List — ordered collection of items

```
columns = ['name', 'age', 'salary', 'city']
columns[0]    # 'name'
columns[-1]   # 'city'
```

Dictionary — key-value pairs

```
employee = {'name': 'Rahul', 'age': 24, 'salary': 35000}
employee['name']    # 'Rahul'
employee['salary']  # 35000
```

Use lists when order matters. Use dictionaries when you need to look up by name.

2.3 If Statements

```
salary = 35000

if salary >= 50000:
    print('Above target')
elif salary >= 30000:
```

```
    print('On track')
else:
    print('Below target')
```

2.4 For Loops

```
cities = ['Delhi', 'Mumbai', 'Pune']

for city in cities:
    print(city)

# Loop through numbers
for i in range(5):
    print(i)    # 0, 1, 2, 3, 4
```

2.5 Functions

```
# Define once, use anywhere
def clean_salary(value):
    value = str(value).replace(',', '').replace('₹', '')
    return float(value)

clean_salary('₹35,000')    # Returns 35000.0
```

Write functions when you find yourself writing the same 3 lines more than once.

Section 3 — NumPy Essentials

NumPy runs behind Pandas. You will not use it directly often but you must understand it.

```
import numpy as np

# Create array
arr = np.array([10, 20, 30, 40, 50])
```

```
# Basic operations
np.mean(arr)      # 30.0
np.median(arr)    # 30.0
np.std(arr)       # 14.14
np.sum(arr)       # 150
np.min(arr)       # 10
np.max(arr)       # 50

# NaN handling
arr2 = np.array([10, 20, np.nan, 40])
np.nanmean(arr2)  # Ignores NaN – returns 23.33
```

Section 4 — Pandas Complete Reference

4.1 Reading Data

```
import pandas as pd

# Read CSV
df = pd.read_csv('sales_data.csv')

# Read Excel
df = pd.read_excel('report.xlsx', sheet_name='Sheet1')

# Read with options
df = pd.read_csv('data.csv', encoding='utf-8', skiprows=2)

# Write output
df.to_csv('output.csv', index=False)
df.to_excel('output.xlsx', index=False)
```

4.2 First Look at Any Dataset

Run these 6 lines on every new file before anything else:

```
df.shape          # (rows, columns)
df.dtypes         # data type of each column
```

```
df.head()           # first 5 rows
df.tail()           # last 5 rows
df.info()           # full summary – nulls, types, memory
df.describe()       # mean, min, max, std for numeric columns
```

4.3 Selecting Data

```
# Select one column
df['salary']

# Select multiple columns
df[['name', 'salary', 'city']]

# Select rows by condition
df[df['salary'] > 50000]
df[df['city'] == 'Delhi']
df[(df['salary'] > 30000) & (df['city'] == 'Mumbai')]

# Select by row number
df.iloc[0]          # first row
df.iloc[0:5]        # first 5 rows
df.iloc[:, 0:3]     # first 3 columns, all rows

# Select by label
df.loc[0, 'name']    # row 0, name column
df.loc[:, 'salary':'city'] # all rows, salary to city
```

4.4 Data Cleaning — Full Workflow

```
# Step 1 – Check nulls
df.isnull().sum()
df.isnull().sum() / len(df) * 100 # percentage missing

# Step 2 – Handle nulls
df.dropna() # remove all rows with nulls
df.dropna(subset=['salary']) # remove only where sa
```

```

lary is null
df['salary'].fillna(0) # fill with zero
df['city'].fillna('Unknown') # fill with placeholder
df['salary'].fillna(df['salary'].mean()) # fill with average

# Step 3 – Remove duplicates
df.duplicated().sum() # count duplicates
df.drop_duplicates() # remove all
df.drop_duplicates(subset=['email']) # remove based on one column

# Step 4 – Fix data types
df['salary'] = df['salary'].astype(int)
df['date'] = pd.to_datetime(df['date'])
df['amount'] = df['amount'].str.replace(',', '').astype(float)

# Step 5 – Clean text
df['name'] = df['name'].str.strip() # remove spaces
df['city'] = df['city'].str.lower() # lowercase
df['city'] = df['city'].str.title() # Title Case
df['phone'] = df['phone'].str.replace(' ', '') # remove spaces

# Step 6 – Clean column names
df.columns = df.columns.str.strip().str.lower().str.replace(' ', '_')

# Step 7 – Reset index after cleaning
df = df.reset_index(drop=True)

```

4.5 GroupBy — Aggregation

```

# Basic groupby
df.groupby('city')['salary'].mean()

```

```

df.groupby('city')['salary'].sum()
df.groupby('city')['salary'].count()

# Multiple aggregations at once
df.groupby('city').agg({
    'salary': ['mean', 'sum', 'count'],
    'age': 'mean'
})

# Group by multiple columns
df.groupby(['city', 'department'])['salary'].mean()

# Reset index after groupby
result = df.groupby('city')['salary'].mean().reset_index()

```

4.6 Merge and Join

```

# INNER JOIN – only matching rows
pd.merge(df1, df2, on='employee_id', how='inner')

# LEFT JOIN – all from left, match from right
pd.merge(df1, df2, on='employee_id', how='left')

# RIGHT JOIN
pd.merge(df1, df2, on='employee_id', how='right')

# FULL OUTER JOIN
pd.merge(df1, df2, on='employee_id', how='outer')

# Join on different column names
pd.merge(df1, df2, left_on='emp_id', right_on='employee_id', how='left')

# Stack rows – same columns, combine datasets
pd.concat([df1, df2], ignore_index=True)

```

4.7 Pivot Tables in Pandas

```

# Basic pivot
df.pivot_table(values='sales', index='region', columns='month', aggfunc='sum')

# Multiple values
df.pivot_table(values=['sales', 'profit'], index='region', aggfunc='sum')

# Fill missing values in pivot
df.pivot_table(values='sales', index='region', columns='month',
                aggfunc='sum', fill_value=0)

```

4.8 String Operations

```

# Most used str functions
df['name'].str.upper()
df['name'].str.lower()
df['name'].str.strip()
df['name'].str.len() # character count
df['name'].str.contains('Kumar') # True/False filter
df['name'].str.startswith('A')
df['name'].str.replace('Ltd', 'Limited')
df['email'].str.split('@').str[0] # extract username
from email
df['phone'].str[-10:] # last 10 digits

```

4.9 Date and Time

```

# Convert to datetime
df['date'] = pd.to_datetime(df['date'])

# Extract components
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month
df['day'] = df['date'].dt.day
df['weekday'] = df['date'].dt.day_name()

```



```

df['quarter'] = df['date'].dt.quarter

# Date calculations
df['days_since'] = (pd.Timestamp.today() - df['date']).dt.days
df['month_end'] = df['date'] + pd.offsets.MonthEnd(0)

# Filter by date
df[df['date'] > '2024-01-01']
df[df['date'].dt.year == 2024]
df[df['date'].dt.month == 3]

```

4.10 Apply and Lambda

```

# Apply a function to a column
df['salary_category'] = df['salary'].apply(lambda x: 'High'
if x > 50000 else 'Low')

# Apply custom function
def categorize(row):
    if row['salary'] > 50000 and row['experience'] > 3:
        return 'Senior'
    else:
        return 'Junior'

df['level'] = df.apply(categorize, axis=1)

# Apply to clean a column
df['phone'] = df['phone'].apply(lambda x: str(x).strip().replace(' ', ''))

```

4.11 Value Counts and Unique

```

df['city'].value_counts()           # frequency count
df['city'].value_counts(normalize=True) # percentage
df['city'].nunique()                # number of unique values

```

```
df['city'].unique() # list of unique va  
lues
```

4.12 Sorting and Ranking

```
df.sort_values('salary', ascending=False) # highes  
t first  
df.sort_values(['city', 'salary'], ascending=[True, False])  
df['rank'] = df['salary'].rank(ascending=False) # add ra  
nk column  
df.nlargest(10, 'salary') # top 10  
df.nsmallest(5, 'salary') # bottom  
5
```

Section 5 — Exploratory Data Analysis (EDA) Workflow

This is the exact process to run on every new dataset before any analysis.

```
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
df = pd.read_csv('data.csv')  
  
# Step 1 – Size and structure  
print(df.shape)  
print(df.dtypes)  
  
# Step 2 – Missing values  
print(df.isnull().sum())  
  
# Step 3 – Duplicates  
print(df.duplicated().sum())  
  
# Step 4 – Basic statistics  
print(df.describe())
```

```
# Step 5 – Distribution of key columns
df['salary'].hist(bins=20)
plt.show()

# Step 6 – Relationships
df[['salary', 'experience', 'age']].corr()
```

Section 6 — Data Visualization

6.1 Matplotlib Basics

```
import matplotlib.pyplot as plt

# Line chart
plt.plot(df['month'], df['sales'])
plt.title('Monthly Sales')
plt.xlabel('Month')
plt.ylabel('Sales')
plt.show()

# Bar chart
plt.bar(df['city'], df['revenue'])
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Histogram
plt.hist(df['salary'], bins=15)
plt.show()

# Save chart
plt.savefig('chart.png', dpi=150, bbox_inches='tight')
```

6.2 Seaborn for Cleaner Charts

```

import seaborn as sns

# Bar plot
sns.barplot(data=df, x='city', y='sales')

# Box plot – shows distribution and outliers
sns.boxplot(data=df, x='department', y='salary')

# Heatmap – correlations
sns.heatmap(df.corr(), annot=True, fmt='.2f', cmap='coolwarm')

# Count plot – frequency
sns.countplot(data=df, x='city')

```

Section 7 — Automation With Python

7.1 Merge Multiple Excel Files

```

import os
import pandas as pd

folder = 'monthly_reports/'
all_files = [f for f in os.listdir(folder) if f.endswith('.xlsx')]

all_data = []
for file in all_files:
    temp = pd.read_excel(folder + file)
    temp['source_file'] = file
    all_data.append(temp)

final = pd.concat(all_data, ignore_index=True)
final.to_excel('combined_report.xlsx', index=False)

```

7.2 Automate a Weekly Report

```

import pandas as pd
from datetime import datetime

df = pd.read_csv('sales.csv')
df['date'] = pd.to_datetime(df['date'])

# Filter this week
today = pd.Timestamp.today()
week_start = today - pd.Timedelta(days=7)
weekly = df[df['date'] >= week_start]

# Summary
summary = weekly.groupby('region').agg(
    total_sales=('sales', 'sum'),
    total_orders=('order_id', 'count'),
    avg_order_value=('sales', 'mean')
).reset_index()

# Export with date in filename
filename = f"weekly_report_{datetime.today().strftime('%Y%m%d')}.xlsx"
summary.to_excel(filename, index=False)
print(f'Report saved: {filename}')

```

Section 8 — Interview Cheat Sheet

What Every DA Interview Tests in Python

Topic	What They Ask	Function to Know
Data loading	Read a CSV or Excel file	read_csv, read_excel
First look	Describe the dataset	shape, info, describe
Null handling	Find and fix missing values	isnull, fillna, dropna
Duplicates	Identify and remove	uplicated, drop_duplicates
Filtering	Subset rows by condition	Boolean indexing
Aggregation	Group and summarize	groupby, agg

Topic	What They Ask	Function to Know
Merging	Combine two datasets	merge, concat
Sorting	Order by a column	sort_values, nlargest
String cleaning	Fix messy text	str.strip, str.lower, str.replace
Date handling	Extract year, month, day	dt.year, dt.month, <u>pd.to_datetime</u>
Visualization	Show a basic chart	bar, hist, lineplot

5 Questions Asked in Almost Every DA Interview

1. How do you handle missing values in a dataset
2. What is the difference between merge and concat in Pandas
3. How do you find duplicate rows and remove them
4. Write a GroupBy query to find average salary by city
5. How do you convert a string column to datetime

Section 9 — Common Errors and Fixes

Error	Cause	Fix
KeyError: 'column_name'	Column does not exist	Check df.columns
ValueError: could not convert string to float	String in numeric column	str.replace then astype
TypeError: unsupported operand	Wrong data type in calculation	Check dtypes, convert first
AttributeError: 'NoneType'	Function returned None	Check if variable is empty
SettingWithCopyWarning	Editing a slice of DataFrame	Use .copy() when slicing

Section 10 — 30-Day Python Learning Plan

Week	Focus	Goal
Week 1	Basics + NumPy	Data types, lists, loops, functions, arrays
Week 2	Pandas Part 1	Read data, select, filter, clean, handle nulls

Week	Focus	Goal
Week 3	Pandas Part 2	GroupBy, merge, pivot, string, date operations
Week 4	Visualization + Automation	Matplotlib, Seaborn, merge files, automate reports

Daily practice rule: Take one real dataset. Apply that week's concept to it. 45 minutes every day beats 5 hours on Sunday.

Final Note

Every concept in this handbook has come from real analyst work or real DA interviews.

Nothing here is theory for theory's sake.

If you are a BBA or BCom fresher reading this — you do not need a computer science degree to use Python for data work. You need this handbook, a real dataset, and 30 days of consistent practice.

The ₹10-12 LPA job is not looking for a Python developer. It is looking for someone who can think with data and use Python to work faster.

That is exactly what this handbook prepares you for.